

Tema 1

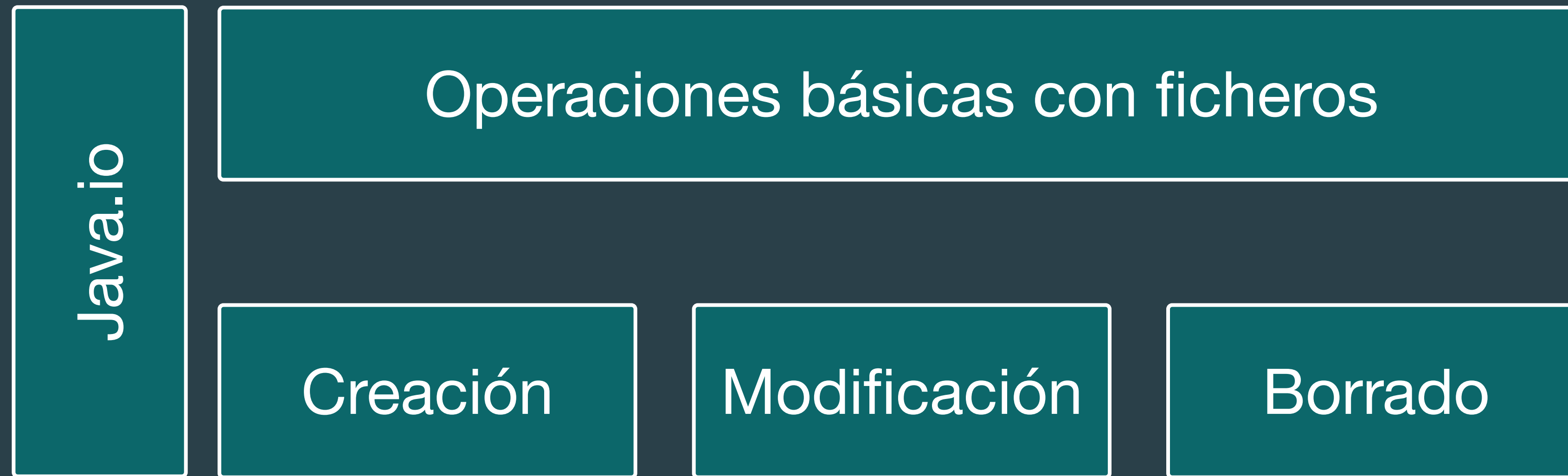
Introducción al manejo de ficheros

Acceso a datos

https://www.w3schools.com/java/java_files.asp

<https://docs.oracle.com/javase/8/docs/api/java/io/File.html>

1. Introducción al manejo de ficheros



2. Definición y tipos de ficheros

Definición

Sucesión de bits almacenada en un fichero

Tipos

ASCII

Líneas de texto en formato ASCII.

Binarios

Información en código binario

2.1. Clase File (Java.io.File)

File	Representación abstracta de un fichero en Java
Constructor	<code>File (String <i>pathname</i>)</code>
Ejemplo	<code>File fichero = new File("/Desktop/file.txt");</code>

Métodos de la clase File

2.1. Clase File (Java.io.File)

Método	<code>[boolean] createNewFile()</code>
Crear fichero	Crea un fichero nuevo y vacío con la ruta indicada en el constructor
Ejemplo	<code>File fichero = new File("/Desktop/file.txt"); fichero.createNewFile();</code>

2.1. Clase File (Java.io.File)

Método	<code>[boolean] delete()</code>
Borrar fichero	Elimina el fichero o directorio definido por la ruta indicada en el constructor
Ejemplo	<code>File fichero = new File("/Desktop/file.txt"); fichero.delete();</code>

2.1. Clase File (Java.io.File)

Método	<code>[boolean] mkdir()</code>
Borrar fichero	Crea el directorio deseado.
Ejemplo	<code>File fichero = new File("/Desktop/folder"); fichero.mkdir();</code>

2.1. Clase File (Java.io.File)

Método	<code>[boolean] mkdirs()</code>
Borrar fichero	Crea el directorio deseado <u>incluyendo cualquier directorio padre no existente</u>
Ejemplo	<code>File fichero = new File("/Desktop/folder"); fichero.mkdirs();</code>

2.1. Clase File (Java.io.File)

Método	<code>[String] getName()</code>
Borrar fichero	Devuelve el pathname.
Ejemplo	<pre>File fichero = new File("/Desktop/folder"); fichero.getName();</pre>

Práctica

Ejercicio 1

Práctica

```
1  //Ejercicio 1
2
3  import java.io.File;
4
5  public static void main(String[] args) {
6      File dir = new File("P1/cine_granada");
7      dir.mkdir();
8  }
```

Práctica

Ejercicio 2

Práctica

```
1  //Ejercicio 2
2
3  ✓ public static void main(String[] args) {
4      String[] dias = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"};
5  ✓  for (String dia : dias) {
6      File dir = new File("P1/" + dia);
7      dir.mkdir();
8  }
9  }
```


2.1. Clase File (Java.io.File)

Método	<code>[boolean] renameTo(File dest)</code>
Borrar fichero	Renombra el fichero. Puede servir para mover archivos.
Ejemplo	<pre>File fichero1 = new File("/Desktop/fichero1.txt"); File fichero2 = new File("/Desktop/fichero2.txt"); fichero1.renameTo(fichero2);</pre>

Práctica

Ejercicio 3

Práctica

```
1  //Ejercicio 3
2
3  public static void main(String[] args) {
4      String[] dias = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"};
5      for (String dia : dias) {
6          File origen = new File("P1/" + dia);
7          File destino = new File("P1/cine_granada/" + dia);
8          origen.renameTo(destino);
9      }
10 }
```

2.1. Clase File (Java.io.File)

Método	<code>[boolean] exists()</code>
Borrar fichero	Comprueba si existe el fichero.
Ejemplo	<pre>File fichero = new File("/Desktop/fichero.txt"); fichero.exists();</pre>

Práctica

Ejercicio 4

Práctica

```
3  //Ejercicio 4.1
4
5  ✓ public static void main(String[] args) {
6      File dir = new File("P1/cine_granada");
7  ✓  if (!dir.exists()) {
8      |     dir.mkdir();
9      |
10     }
11
12  //Ejercicio 4.2
13
14  ✓ public static void main(String[] args) {
15      String[] dias = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"};
16  ✓  for (String dia : dias) {
17      |     File dir = new File("P1/" + dia);
18  ✓  |     if (!dir.exists()) {
19      |         dir.mkdir();
20      |     }
21      |
22  }
```

2.1. Clase File (Java.io.File)

Ruta absoluta

Ruta relativa

Ruta completa del recurso desde el directorio raíz

Representa una parte de la ruta, se tiene en cuenta el directorio actual

c:/usuario/data/desktop/img.png

desktop/img.png

c:

usuario

data



desktop

img.png

2.1. Clase File (Java.io.File)

Método	<code>[String] getPath()</code>
Borrar fichero	Devuelve la ruta relativa.
Ejemplo	<pre>File fichero = new File("/Desktop/folder"); fichero.getPath();</pre>

2.1. Clase File (Java.io.File)

Método	<code>[String] getAbsolutePath()</code>
Borrar fichero	Devuelve la ruta absoluta.
Ejemplo	<pre>File fichero = new File("/Desktop/folder"); fichero.getAbsolutePath();</pre>

2.1. Clase File (Java.io.File)

Método	<code>[String] getParent()</code>
Borrar fichero	Devuelve el pathname del directorio superior (padre).
Ejemplo	<code>File fichero = new File("/Desktop/folder"); fichero.getParent();</code>

Práctica

Ejercicio 5

Práctica

```
1  //Ejercicio 5
2
3  public static void main(String[] args) {
4
5      File dirCine = new File("P1/cine_granada");
6      if (!dirCine.exists()) {
7          dirCine.mkdir();
8          System.out.println("Se ha creado el directorio con ruta absoluta: " + dirCine.getAbsolutePath());
9      }
10
11     String[] dias = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"};
12     File dir = new File("P1");
13     for (String dia : dias) {
14         dir = new File("P1/" + dia);
15         if (!dir.exists()) {
16             dir.mkdir();
17             System.out.println("Se ha creado el directorio con ruta absoluta: " + dir.getAbsolutePath());
18         }
19     }
20 }
```

2.1. Clase File (Java.io.File)

Método	<code>[File[]] listFiles()</code>
Borrar fichero	Devuelve un array de Files que representan los ficheros del directorio indicado.
Ejemplo	<code>File fichero = new File("/Desktop/folder"); fichero.listFiles();</code>

Práctica

Ejercicio 6

Práctica

```
1  import java.io.File;
2
3  public static void main(String args[]) {
4
5      File directorio = new File("/P1/cine_granada");
6      System.out.println("Archivos creados hasta ahora:");
7      if (directorio.exists()){
8          File[] lista = directorio.listFiles();
9          for (File elemento : lista){
10             System.out.println("Ruta relativa:" + elemento.getPath());
11          }
12      }
13 }
```

2.1. Clase File (Java.io.File)

Método	<code>[boolean] canWrite()</code> <code>[boolean] canRead()</code>
Borrar fichero	Comprueba si se puede escribir (write) o leer (read) el archivo.
Ejemplo	<code>File fichero = new File("/Desktop/fichero.txt");</code> <code>fichero.canWrite();</code> <code>fichero.canRead();</code>

Práctica

Ejercicio 7

Práctica

```
1 //Ejercicio 7
2
3 public static void main(String[] args){
4     String[] dias = {"Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado", "Domingo"};
5     for (String dia : dias) {
6         File archivo = new File("P1/cine_granada/" + dia + "/sesiones.txt");
7         archivo.createNewFile();
8     }
9 }
```


3. Formas de acceder a un fichero

Secuencial	Accedemos al fichero carácter a carácter (o byte a byte) de forma ordenada desde el inicio hasta el final.
Aleatorio	Nos permite acceder a posiciones concretas de nuestro fichero

Acceso secuencial a ficheros mediante bytes

3.1. Lectura

Clase	FileInputStream https://docs.oracle.com/javase/8/docs/api/java/io/FileInputStream.html
Constructor	FileInputStream(String name) o FileInputStream(File file)
Uso	Leer datos en bruto como bytes desde un archivo. Es ideal para leer archivos binarios.

3.1. Lectura

Método	<code>[int] read()</code>
Leer	Lee un byte del fichero (en orden). Devuelve -1 si no quedan datos que leer.
Ejemplo	<pre>FileInputStream entrada = new FileInputStream("desktop/fichero.bin"); entrada.read(); entrada.close();</pre>

3.2. Escritura

Clase	FileOutputStream https://docs.oracle.com/javase/8/docs/api/java/io/FileOutputStream.html
Constructor	FileOutputStream(String name) o FileOutputStream(File file)
Uso	Escribir datos en bruto como bytes a un archivo. Es ideal para escribir archivos binarios.

3.2. Escritura

Método	<code>write(byte[] b)</code>
Escribir	Escribe un conjunto de bytes en un fichero.
Ejemplo	<pre>FileOutputStream output = new FileOutputStream("desktop/fichero.txt"); String cadena = "prueba de escritura"; byte [] arrayBytes = cadena.getBytes(); output.write(arrayBytes); output.close();</pre>

Práctica

Ejercicio 8

Práctica

```
1  //Ejercicio 8
2
3  import java.io.File;
4  import java.io.FileInputStream;
5  import java.io.FileOutputStream;
6
7  public static void main(String[] args){
8      File archivo = new File("P1/cine_granada/Lunes/sesiones.txt");
9
10     FileOutputStream fos = new FileOutputStream(archivo);
11     String texto = "Spiderman (2002): 18:00 - 20:07";
12     fos.write(texto.getBytes());
13
14     FileInputStream fis = new FileInputStream(archivo);
15     int i;
16     while ((i = fis.read()) != -1) {
17         System.out.print((char) i);
18     }
19 }
```

Acceso secuencial a ficheros mediante caracteres

3.3. Lectura

Clase	FileReader https://docs.oracle.com/javase/8/docs/api/java/io/FileReader.html
Constructor	FileReader(String name) o FileReader(File file)
Uso	Leer datos de caracteres de un archivo. Es adecuado para leer archivos de texto como .txt, .xml, .json, etc.

3.3. Lectura

Método	<code>[int] read()</code>
Leer	Lee un carácter del fichero (en orden). Devuelve -1 si no quedan datos que leer.
Ejemplo	<pre>FileReader entrada = new FileReader("desktop/fichero.txt") char datos = (char)entrada.read(); entrada.close();</pre>

3.4. Escritura

Clase	FileWriter https://docs.oracle.com/javase/8/docs/api/java/io/FileWriter.html
Constructor	FileWriter(String name) o FileWriter(File file)
Uso	Escribir datos de caracteres en un archivo. Es adecuado para escribir archivos de texto como .txt, .xml, .json, etc.

3.4. Escritura

Método	<code>write()</code>
Escribir	Escribe un conjunto de caracteres en un fichero.
Ejemplo	<pre>FileWriter escritor = new FileWriter("desktop/fichero.txt"); escritor.write("texto de ejemplo a escribir"); escritor.close();</pre>

Práctica

Ejercicio 9

Práctica

```
1  //Ejercicio 9
2
3  import java.io.File;
4  import java.io.FileReader;
5  import java.io.FileWriter;
6
7  public static void main(String[] args){
8      File archivo = new File("P1/cine_granada/Martes/sesiones.txt");
9
10     FileWriter fw = new FileWriter(archivo);
11     fw.write("Iron Man (2008): 17:00 - 19:06");
12
13     FileReader fr = new FileReader(archivo);
14     int i;
15     while ((i = fr.read()) != -1) {
16         System.out.print((char) i);
17     }
18 }
```

Acceso aleatorio a ficheros mediante bytes

4. Acceso aleatorio basado en bytes

RandomAccessFile

Lectura - Escritura

Acceso a un punto
determinado del fichero

<https://docs.oracle.com/javase/8/docs/api/java/io/RandomAccessFile.html>

4. Acceso aleatorio basado en bytes

Constructor

RandomAccessFile(String path, string mode)

RandomAccessFile(File file, string mode)

Modos de acceso

r

Modo lectura

rw

Modo lectura y escritura

rwd

Lectura y escritura síncrona

No garantiza actualización de metadatos

rws

Lectura y escritura síncrona

Garantiza actualización de metadatos

4. Acceso aleatorio basado en bytes

seek (long position)

Permite posicionarnos en el punto que indiquemos en el fichero

```
RandomAccessFile file = new RandomAccessFile(String path, string mode)
```

```
file.seek(12);
```


4. Acceso aleatorio basado en bytes

```
RandomAccessFile file = new RandomAccessFile(String path, string mode);
```

```
file.read( );  
//Permite leer un Byte donde está colocado el puntero
```

```
file.readLine( );  
//Permite leer la siguiente línea de Bytes a partir de donde está colocado el  
puntero
```


4. Acceso aleatorio basado en bytes

```
RandomAccessFile file = new RandomAccessFile(String path, string mode);
```

```
file.write("Example".getBytes());  
//Permite escribir una cadena de bytes donde está colocado el puntero  
//El puntero avanza tras escribir
```

```
file.writeBytes("Example");  
//Permite escribir un String como una secuencia de Bytes
```

Práctica

Ejercicio 10

Práctica

```
1  //Ejercicio 10
2
3  import java.io.File;
4  import java.io.RandomAccessFile;
5
6  public static void main(String[] args) {
7      File archivo = new File("P1/cine_granada/Miércoles/sesiones.txt");
8
9      //Escribir
10     RandomAccessFile raf = new RandomAccessFile(archivo, "rws");
11     raf.writeBytes("Titanic (1998): 17:00 - 20:15");
12
13     //Modificar
14     raf.seek("Titanic (".length()); // posicionarse justo donde comienza el año
15     raf.writeBytes("1997");
16
17     //Leer
18     String linea;
19     while ((linea = raf.readLine()) != null) {
20         System.out.println(linea);
21     }
22 }
```


Agradecimientos

**Material adaptado a partir del trabajo de:
Antonio Cuadros Lapresa**

*“Lo mejor que se puede compartir
es el conocimiento” - Alain Ducasse*